SIMATS ENGINEERING

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA4712 – DEEP LEARNING CO1 – ASSESSMENT 5

GENERATIVE MODELLING, SAMPLING AND EVALUATION ENGINEERING

Student Name	**B Akshaya**
Registration Number	**192424336**
Submission Date	**29.10.2026**
Scenario	**Healthcare risk prediction**

Assignment overview

This assignment explains the mathematical and machine-learning choices behind a hospital disease-risk prediction system that uses patient age, blood pressure, glucose level, cholesterol and family history. Each task is presented as connected technical discussion with a worked calculation or data table, a Python code excerpt and a notebook-style output panel. The code excerpts can be executed in Google Collab. Panels for Tasks 1 to 6 and 9 are computed directly from the numbers given in the questions; panels for Tasks 7, 8 and 10 to 12 come from clearly labelled synthetic simulations and are not hospital data. All predictions are decision support for clinicians and never a diagnosis.

TASK A1 – Formulate an Energy-Based Generative Model (4 Marks)

A1.1 – Define the Generative Requirement (1 Mark)

The model learns an approximate data distribution p_data(v), rather than only a decision boundary. A good sample should be structurally plausible, consistent with common patterns in the approved data, diverse across repeated generations, useful for downstream algorithm development, and not a near-copy of a training record.

Two differences: (1) a classifier estimates p(y|x) or a label, while a generative model estimates or samples from p(x) or p(x,y); (2) a classifier is judged mainly by predictive metrics, while a generator also needs fidelity, diversity, utility and memorisation/privacy checks.

A1.2 – Derive the Restricted Boltzmann Machine (2 Marks)

Energy function:

Joint distribution:

Because hidden units are conditionally independent given v:

Similarly, visible units are conditionally independent given h:

$p(v_i=1 \mid h) = \sigma(a_i + \Sigma_j W_{ij} h_j)$.

The factorisation follows from the bipartite structure: once the units on one side are fixed, there are no within-side connections coupling the conditional probabilities.

A1.3 – Quantify Intractability (1 Mark)

```
visible_units = 1024

hidden_units = 256

joint_binary_units = visible_units + hidden_units

joint_states = 2 ** joint_binary_units

# Exact integer has 386 decimal digits; scale is about 10^385.

print("Joint states = 2^", joint_binary_units)
```

TASK A2 – Design Contrastive Divergence Learning (4 Marks)

A2.1 – Derive the Learning Gradient (1 Mark)

For one observed data vector v, the log-likelihood gradient for a weight is:

The positive phase increases weights that support associations observed in the data. The negative phase decreases associations that the model generates too often. The exact model expectation is expensive because it depends on Z and the full model distribution.

A2.2 – Design CD-k Pseudocode (2 Marks)

```
ALGORITHM: MINI-BATCH CONTRASTIVE DIVERGENCE CD-k

INPUT: training set D, weights W, biases a and b, learning rate η, steps k

FOR each epoch:
  FOR each mini-batch B in D:
    positive_statistics ← 0
    negative_statistics ← 0
    FOR each data vector v0 in B:
      sample h0 from p(h | v0)
      set v ← v0
      REPEAT k times:
        sample h from p(h | v)
        sample v from p(v | h)
      END REPEAT
      sample hk from p(h | vk)
      accumulate v0 × p(h=1 | v0) into positive_statistics
      accumulate vk × p(h=1 | vk) into negative_statistics
    END FOR
    W ← W + η × (positive_statistics − negative_statistics)/|B|
    a ← a + η × (mean(B) − mean(reconstructed visible samples))
    b ← b + η × (mean(positive hidden probabilities)
            − mean(final hidden probabilities))
  END FOR
END FOR

OUTPUT: trained W, a, b

PERSISTENT CD MODIFICATION:
  Maintain a persistent visible/hidden chain across mini-batches instead of
  restarting each negative chain at a training example.
```

CD-k is an approximation: the negative chain starts at a data point and runs for k Gibbs transitions. Larger k can improve the negative-phase approximation but costs more computation; it does not guarantee convergence to the equilibrium distribution.

A2.3 – Trace One Numerical Update (1 Mark)

TASK A3 – Design a Deep Belief Network and Deep Boltzmann Model (4 Marks)

A3.1 – Compare the Structures (1 Mark)

A Deep Belief Network (DBN) is commonly trained by stacking RBMs. In a standard DBN, the top two layers form an undirected associative model, while lower layers are directed downward during generation. A Deep Boltzmann Machine (DBM) is an undirected model with interactions between adjacent layers throughout the network.

DBN (schematic):

Visible v ← directed generative connection ← Hidden h1

↑ recognition/training representation

Hidden h1 ↔ Hidden h2 (top-level undirected RBM)

DBM (schematic):

Visible v ↔ Hidden h1 ↔ Hidden h2 ↔ Hidden h3

(all adjacent-layer connections are undirected)

A DBN can generate by sampling the top-level model and then propagating downward through directed connections. A DBM requires approximate inference because each hidden layer depends on neighbouring layers on both sides; mean-field inference or MCMC is often used.

A3.2 – Design Layer-Wise Training and Generation (2 Marks)

ALGORITHM: GREEDY LAYER-WISE DBN TRAINING

INPUT: training examples X, number of layers L = 3

set representation R1 ← X

FOR layer ℓ = 1 TO L:

train RBMℓ using Rℓ as visible training data

compute hidden probabilities Pℓ ← p(hℓ=1 | Rℓ)

set Rℓ+1 ← Pℓ for the next layer

END FOR

use the top RBM as the associative model

GENERATION:

initialise a top-layer state

perform Gibbs sampling in the top-level RBM

sample or compute hidden/visible states according to the chosen procedure

propagate downward through trained generative connections

OUTPUT the visible sample

Sampling draws a binary state from a Bernoulli distribution using its probability. Replacing a sampled state with its probability produces a deterministic expected activation. These are not identical: probabilities reduce sampling noise, while samples represent stochastic generation.

A3.3 – Justify Depth (1 Mark)

Greedy layer-wise training lets each RBM learn a representation before the next layer is trained. Under the standard DBN construction assumptions, adding a layer can improve a variational lower bound on the data log-likelihood. More layers may learn increasingly abstract structure, but also increase computation and can complicate sampling. A DBM differs because inference is not a simple one-way pass; its undirected dependencies require iterative approximate inference.

TASK A4 – Design a Convolutional Boltzmann Model (4 Marks)

A4.1 – Specify the Convolutional Structure (2 Marks)

For filter k at location (r,c), the hidden activation probability can be written p(h_krc=1|v)=σ(b_k + Σ_{u=0..4}Σ_{w=0..4} W_kuw v_(r+u,c+w)). Visible conditional probabilities combine contributions from the filters that overlap each pixel, then apply a sigmoid. Probabilistic pooling groups nearby hidden activations and samples a pooled state so that nearby detections compete or share probability mass.

Weight sharing means the same 5 × 5 pattern detector is used across locations. This reduces parameters and captures translation-related local patterns.

A4.2 – Quantify Parameter Economy (1 Mark)

Hidden positions per filter = 28 × 28 = 784. With 12 filters, total hidden positions = 784 × 12 = 9,408. Shared filter weights = 12 × 5 × 5 = 300, plus 12 filter biases (and visible biases, if used).

A fully connected RBM with 1,024 visible units and 9,408 hidden units would require 1,024 × 9,408 = 9,633,792 connection weights, excluding biases. The convolutional model uses only 300 shared filter weights, plus biases, so it is dramatically more parameter-efficient.

A4.3 – Judge Suitability (1 Mark)

Convolution exploits local spatial structure: a feature such as a small occupied region can be detected wherever it appears. A fully connected RBM does not explicitly exploit this spatial locality and needs separate weights for each visible-hidden pair. A limitation is that local receptive fields may not capture long-range relationships or global layout well without deeper layers or additional global connections.

TASK A5 – Design Directed Latent and Sequential Generators (4 Marks)

A5.1 – Design a Directed Latent-Variable Model (2 Marks)

Using any qφ(z|x), the evidence lower bound (ELBO) is:

The first term is the reconstruction term: it rewards the decoder for explaining the observed sample. The second is the KL divergence penalty: it encourages the encoded latent distribution to remain close to the prior. Maximising ELBO balances reconstruction and a regular latent space.

A5.2 – Explain Reparameterisation (1 Mark)

Directly sampling z from a Gaussian whose mean and variance depend on network parameters creates a stochastic operation that ordinary backpropagation cannot differentiate through directly. The reparameterisation trick writes:

The randomness is isolated in ε, which is independent of μ and σ. Gradients can therefore flow through μ and σ using standard backpropagation.

A5.3 – Design Sequential Generation (1 Mark)

For a sequence of 24 hourly ward states x1,…,x24, the chain rule gives:

Each conditional model predicts the distribution of the next hour given the preceding states. During teacher-forced training, true previous states are provided. During generation, the model's own generated states are fed back, so errors can accumulate along the sequence. Training and evaluation should measure both one-step prediction and full-sequence plausibility.

TASK A6 – Design the Evaluation Before Training (5 Marks)

A6.1 – Explain Likelihood Comparison (1 Mark)

For RBMs and DBNs, exact likelihood depends on a partition function that is usually intractable. Annealed importance sampling (AIS) estimates the partition function by bridging from a tractable reference distribution to the target model through intermediate distributions. The estimate has sampling variance and must be reported with its settings. A VAE's ELBO is a lower bound, not generally the exact log-likelihood; it should not be compared naively with an AIS estimate without clearly stating the different objectives and estimator uncertainty.

A6.2 – Design a Multi-Measure Protocol (2 Marks)

A6.3 – Design a Memorisation Test (1 Mark)

Generate at least 1,000 samples after training. For each generated sample, calculate its minimum Hamming distance to every training sample and to a held-out set. Flag exact duplicates (distance 0) and inspect a small set of closest pairs. Evidence of memorisation includes an unexpectedly high exact-duplicate rate or generated samples consistently much closer to training records than to held-out records. This test cannot prove privacy; additional privacy analysis may be required.

A6.4 – Predict Before Building (1 Mark)

These are hypotheses to record before implementation, not measured results. The experiment should later compare them using the same dataset split and consistent measures.

PART B – IMPLEMENTATION & GENERATIVE INVESTIGATION (25 MARKS)

The source assignment requires real code, test inputs, actual outputs, instrumentation and generated samples. The following sections provide an implementation framework and a reproducible experiment plan. Dataset-dependent result tables must be populated after execution; illustrative or predicted values are not reported as measured results.

TASK B1 – Implement and Instrument the RBM (4 Marks)

B1.1 – Implement and Instrument Learning (2 Marks)

The following compact NumPy implementation uses CD-k updates without automatic differentiation. It records reconstruction error, mean hidden activation, weight norm and a free-energy gap after each epoch. Input X must contain binarised samples with shape (n_samples, n_visible).

```
import numpy as np

def sigmoid(x):

    x = np.clip(x, -30, 30)

    return 1.0 / (1.0 + np.exp(-x))

class RBM:

    def __init__(self, n_visible, n_hidden, lr=0.05, seed=7):

        rng = np.random.default_rng(seed)

        self.rng = rng

        self.W = rng.normal(0, 0.01, (n_visible, n_hidden))

        self.a = np.zeros(n_visible)

        self.b = np.zeros(n_hidden)

        self.lr = lr

    def ph(self, v):

        return sigmoid(v @ self.W + self.b)

    def pv(self, h):

        return sigmoid(h @ self.W.T + self.a)

    def free_energy(self, v):

        visible_term = v @ self.a

        hidden_term = np.logaddexp(0, v @ self.W + self.b).sum(axis=1)

        return -visible_term - hidden_term

    def fit(self, X, epochs=20, batch_size=32, k=1):

        history = []

        for epoch in range(epochs):

            self.rng.shuffle(X)

            rec, acts = [], []

            for start in range(0, len(X), batch_size):

                v0 = X[start:start+batch_size]

                ph0 = self.ph(v0)

                h = (self.rng.random(ph0.shape) < ph0).astype(float)
```

```
            vk = v0.copy()
            for _ in range(k):
                pv = self.pv(h)
                vk = (self.rng.random(pv.shape) < pv).astype(float)
                ph = self.ph(vk)
                h = (self.rng.random(ph.shape) < ph).astype(float)
            phk = self.ph(vk)
            n = len(v0)
            self.W += self.lr * (v0.T @ ph0 - vk.T @ phk) / n
            self.a += self.lr * np.mean(v0 - vk, axis=0)
            self.b += self.lr * np.mean(ph0 - phk, axis=0)
            rec.append(np.mean((v0-vk)**2))
            acts.append(np.mean(ph0))
        train_fe = float(np.mean(self.free_energy(X)))
        history.append({
            "epoch": epoch+1,
            "reconstruction_error": float(np.mean(rec)),
            "mean_hidden_activation": float(np.mean(acts)),
            "weight_norm": float(np.linalg.norm(self.W)),
            "free_energy_train": train_fe
        })
    return history
# Example use:
# X_train and X_test must be binarised arrays, not arbitrary continuous data.
# rbm = RBM(X_train.shape[1], n_hidden=128)
# history = rbm.fit(X_train.copy(), epochs=20, batch_size=32, k=1)
# held_fe = np.mean(rbm.free_energy(X_test))
# free_energy_gap = held_fe - history[-1]["free_energy_train"]
# print(history[-1], "free_energy_gap =", free_energy_gap)
```

Interpretation: lower reconstruction error indicates better one-step reconstruction, but not necessarily better sampling. A rising weight norm can indicate unstable learning. A large train–held-out free-energy gap may indicate poor generalisation, although free energy is not itself a complete sample-quality metric. Record the dataset, seed, preprocessing and hardware alongside results.

B1.2 – Validate Against Exact Computation (2 Marks)

Use a tiny RBM with 6 visible and 3 hidden units: there are 2^9 = 512 joint states, so exact enumeration is practical. For each visible vector, enumerate all hidden states to compute Z and exact log p(v). Compare a finite-difference or exact gradient with the CD estimate.

```
from itertools import product

import numpy as np

def all_binary(n):

    return np.array(list(product([0.0, 1.0], repeat=n)))

def exact_partition(rbm):

    V = all_binary(rbm.W.shape[0])

    H = all_binary(rbm.W.shape[1])

    energies = []

    for v in V:

        for h in H:

            E = -(v @ rbm.a) - (h @ rbm.b) - (v @ rbm.W @ h)

            energies.append(E)

    energies = np.asarray(energies)

    m = np.max(-energies)

    return float(np.exp(-energies-m).sum() * np.exp(m))

def exact_log_prob_visible(rbm, v):

    H = all_binary(rbm.W.shape[1])

    scores = np.array([-( -(v @ rbm.a) - (h @ rbm.b) - (v @ rbm.W @ h) )

                       for h in H])

    m = scores.max()

    log_unnormalised = m + np.log(np.exp(scores-m).sum())

    return log_unnormalised - np.log(exact_partition(rbm))

# Tiny validation setup:

# small = RBM(6, 3, lr=0.01, seed=1)

# print("Z =", exact_partition(small))

# print("log p(v) =", exact_log_prob_visible(small, X_small[0]))
```

Boundary case: train on repeated identical patterns. The code should execute without NaN/Inf, but generated diversity may collapse. This checks numerical stability and whether the evaluation protocol detects mode collapse. Exact and CD gradients need not match exactly; CD is biased, so compare direction and magnitude over multiple seeds rather than expecting equality.

TASK B2 – Investigate Contrastive Divergence Behaviour (4 Marks)

B2.1 – Controlled Experiments (2 Marks)

B2.2 – Predict and Compare (1 Mark)

Prediction before running: increasing k should increase computation per update and may improve the negative-phase approximation; sample quality may improve and then plateau, but need not improve monotonically. Persistent chains may better track the model distribution, but can mix slowly. After execution, compare each prediction with the measured quality and time values and state where the prediction was supported or contradicted.

B2.3 – Interrogate Reconstruction Error (1 Mark)

Reconstruction error is insufficient by itself. A model can reconstruct training patterns accurately while generating only a few common patterns, failing to cover minority modes, or copying near-duplicates. Therefore, report reconstruction error together with held-out energy/likelihood estimates where available, sample diversity, nearest-neighbour memorisation checks and downstream utility.

TASK B3 – Build and Generate with a Deep Belief Network (4 Marks)

B3.1 – Stack and Generate (2 Marks)

Train three RBMs layer-wise: the first on X, the second on the first layer's hidden activations, and the third on the second layer's activations. Use the top RBM for Gibbs sampling, then propagate downward to create visible samples. Generate at least 64 samples and save a grid of outputs. Record top-layer Gibbs steps and show mixing evidence such as class/cluster coverage or transitions between distinct sample groups.

B3.2 – Compare Depth (1 Mark)

Choose depth using held-out evidence, not training reconstruction alone. Deeper DBNs may model more abstract structure but can take longer to train and sample and may still mix poorly.

B3.3 – Test Generative Pre-Training (1 Mark)

Use exactly 120 labelled examples for the low-label experiment. Compare the same classifier architecture with (i) weights initialised from DBN pre-training and (ii) random initialisation. Use identical train/validation splits and optimisation settings. Report accuracy or F1, learning curves and epochs to convergence. A gain is supported only if validation performance or training stability improves; the result is dataset-dependent and must be measured.

TASK B4 – Implement a Convolutional Boltzmann Model (4 Marks)

B4.1 – Implement the Model (2 Marks)

Implement a small convolutional RBM with 32 × 32 binarised inputs, 12 filters of size 5 × 5 and valid convolution. A simplified implementation may use independent hidden Bernoulli units and omit probabilistic max-pooling, but must explicitly state this change: it no longer models competition among nearby hidden activations. This simplification is acceptable for an initial engineering prototype, not an exact reproduction of a pooled convolutional RBM.

For each training batch: compute convolutional hidden probabilities, sample hidden states, reconstruct visible probabilities using the transposed convolution, sample reconstructed visible states, and update shared filters using positive-minus-negative associations. Save learned filters and at least 64 generated samples.

B4.2 – Compare with a Fully Connected RBM (1 Mark)

Compare both models on the same binarised dataset and training budget. The convolutional model has 300 shared filter weights for the specified filters, while a fully connected RBM with 9,408 hidden units has 9,633,792 connection

weights. Compare actual time per epoch, reconstruction error, diversity and visual plausibility. Parameter count predicts efficiency but does not alone establish sample quality.

B4.3 – Interpret the Learned Filters (1 Mark)

Display the 12 learned 5 × 5 filters as grayscale tiles. Filters with concentrated positive/negative regions may respond to local edges, corners or occupancy blobs. Interpret these patterns only after visual inspection of the learned filters. A possible limitation is that samples contain locally plausible regions but inconsistent global structure; confirm this from generated outputs rather than claiming it without evidence.

TASK B5 – Implement Directed and Sequential Generators (4 Marks)

B5.1 – Implement the Variational Model (2 Marks)

Implement a VAE with an encoder producing μ and log σ², latent dimension z_dim, and a decoder producing Bernoulli probabilities for binarised images. Use the reparameterisation trick. For every epoch, record ELBO, reconstruction term and KL divergence. For a Bernoulli decoder, reconstruction log-likelihood is the sum of binary cross-entropies with sign reversed; the KL term for a diagonal Gaussian can be computed analytically.

```
# PyTorch-style VAE loss (assumes recon_logits, x, mu, logvar)

import torch

import torch.nn.functional as F

def vae_loss(recon_logits, x, mu, logvar, beta=1.0):

    # Sum over pixels, mean over batch

    recon_nll = F.binary_cross_entropy_with_logits(

        recon_logits, x, reduction="none"

    ).flatten(start_dim=1).sum(dim=1).mean()

    kl = (-0.5 * (1 + logvar - mu.pow(2) - logvar.exp())

          .sum(dim=1).mean())

    total = recon_nll + beta * kl

    elbo = -(recon_nll + kl)

    return total, elbo, recon_nll, kl
```

Expected trend: reconstruction loss should generally decrease early; KL should remain meaningfully positive if the latent variable is used. A near-zero KL combined with poor latent sensitivity can indicate posterior collapse. Record actual curves; do not assume the expected trend occurred.

B5.2 – Investigate the Latent Space (1 Mark)

B5.3 – Generate Sequentially (1 Mark)

For the row-wise generator, predict each row or each next categorical state conditioned on previously generated rows/states. Compare it with the VAE using the same held-out dataset. Record sample-generation time and prediction error at each position/time step. Autoregressive generation can preserve local conditional dependencies but is sequential and errors can accumulate; a VAE can generate in one decoder pass but may be less precise for long structured dependencies.

TASK B6 – Evaluate, Stress and Make the Engineering Decision (5 Marks)

B6.1 – Evaluate All Models (1 Mark)

Use a common protocol and report four measures for all five models. Recommended measures are: (1) fidelity, using held-out likelihood/AIS where applicable or a clearly specified feature-distribution metric; (2) diversity, using unique rate or mean pairwise Hamming distance; (3) memorisation, using exact-match rate and nearest-training distance; and (4) utility, using downstream performance when trained on synthetic data and tested on held-out real data. Record time per epoch as an additional engineering metric.

B6.2 – Test Memorisation and Diversity (1 Mark)

Select the two strongest models based on the pre-declared evaluation protocol. For each, generate the same number of samples and compute the fraction exactly matching any training record, the minimum Hamming distance to training data, and pairwise generated-sample distance. Report numerical values and inspect closest pairs. If exact duplicates or unusually close training neighbours are common, investigate memorisation before release.

B6.3 – Engineer a Failure (1 Mark)

Chosen failure test: deliberately reduce Gibbs steps to k=1 and use a high learning rate compared with the stable baseline. Prediction before run: samples may become repetitive or unstable, while weights may grow quickly. Diagnosis after run must use measured reconstruction error, weight norm, free-energy gap and diversity. If these symptoms are not observed, report the actual outcome rather than forcing the prediction.

B6.4 – Write the Engineering Decision Note (1 Mark)

Provisional recommendation: select the model that best balances held-out fidelity, sample diversity, downstream utility, low memorisation evidence and acceptable compute cost. For 32 × 32 binary maps, a convolutional RBM is attractive when local spatial patterns dominate; a VAE is attractive when efficient sampling and a smooth latent space are important; an autoregressive model is attractive when conditional sequence structure is essential. The final deployment choice must follow the measured results table.

Evaluation cannot guarantee that synthetic data is anonymous or free of all privacy leakage. Before release, monitor exact duplicates, nearest-neighbour similarity, rare-group representation, downstream utility, distribution drift and any evidence of identifiable patterns. Apply privacy review and data-governance approval.

B6.5 – Adapt to an Unseen Requirement (1 Mark)

For class-controlled generation, condition the selected generator on a requested class label y. For a VAE, concatenate a one-hot class vector to the encoder/decoder inputs, or train a conditional decoder $p(x|z,y)$. For a five-state categorical sequence, define states $s1,\ldots,s5$ and sample:

CONCLUSION

Generative modelling requires more than producing plausible-looking samples. RBMs learn through energy-based distributions and approximate sampling; DBNs stack learned representations; convolutional RBMs exploit spatial locality; VAEs use latent-variable inference and an ELBO objective; autoregressive generators model sequential conditional distributions. Reliable selection requires consistent evaluation of fidelity, diversity, utility, computation and memorisation risk. The final model should be deployed only after dataset-specific experiments and privacy checks support the decision.